

Robótica II

Proyecto de Cátedra:

**“Robot serie manipulador de vaso de
agua”**

Profesores

Ing. Haarth, Roberto

Ing. Garrido, Gonzalo

Alumnos - Grupo 4

Borquez, Juan	13567
Dalessandro, Francisco	13318
Panonto, Valentín	13583
Roby Culasso, Martina	13510

Índice

1. Resumen	2
2. Introducción	2
3. Objetivos del proyecto	3
3.1. Objetivos generales	3
3.2. Objetivos particulares	3
4. Cinemática	3
5. Dinámica	5
5.1. Definición de la trayectoria	5
5.2. Modelo dinámico utilizado	5
5.3. Cálculo mediante Newton–Euler recursivo	6
5.4. Dinámica directa y validación	6
6. Diseño mecánico	8
6.1. Descripción de eslabones	8
6.2. Eector final	11
6.3. Materiales empleados en la construcción	12
7. Selección de actuadores	16
8. Programación	17
8.1. Controlador local	17
8.2. Autómata para sincronización de actividades de robot serie y robot móvil	20
9. Funcionamiento	22
10. Conclusiones	22

1. Resumen

El presente trabajo describe el diseño, modelado, implementación y validación de un manipulador robótico serie de tres grados de libertad actuados y tres grados de libertad no actuados de una articulación esférica, destinado a manipular un vaso con agua de manera estable, precisa y coordinada con un robot móvil. El proyecto integra aspectos de cinemática, dinámica, diseño estructural, selección de actuadores y desarrollo de un sistema de control distribuido basado en comunicación inalámbrica y coordinación mediante un autómatas central.

Se desarrolló un modelo cinemático del robot - tanto reducido como completo - utilizando parámetros DH, y se realizó un análisis dinámico utilizando el Robotics Toolbox de Peter Corke. Este análisis permitió dimensionar las relaciones de transmisión y verificar la capacidad de los actuadores frente a las exigencias de torque en trayectorias exigentes. La estructura mecánica fue diseñada en CAD y fabricada principalmente mediante impresión 3D, optimizando masa y rigidez.

El sistema de control se implementó en una ESP32 C3 Supermini para el brazo y en una Raspberry Pi Zero actuando como autómatas, utilizando el protocolo MQTT para la coordinación con el robot móvil. Se desarrolló una interfaz HMI en Node-RED para control manual, carga de trayectorias y monitoreo en tiempo real.

Los ensayos de validación muestran que el manipulador cumple los requisitos funcionales impuestos por la cátedra, logrando manipular un vaso de 120 cc sin derrames y coordinando correctamente con el robot móvil durante las secuencias de entrega y devolución.

2. Introducción

La manipulación segura de objetos frágiles o inestables es un desafío central en robótica de servicio y prototipado académico. En este contexto, la cátedra de Robótica II propone el desarrollo de un manipulador serie capaz de tomar y transportar un vaso con agua, integrando conceptos de cinemática, dinámica, diseño estructural, selección de actuadores y control. El proyecto aquí presentado tuvo por objetivo diseñar, fabricar y validar un robot serie antropomórfico de tres grados de libertad actuados, con una muñeca pasiva esférica, que pueda manipular objetos en coordinación con un robot móvil.

El desarrollo se abordó de manera integral: mediante Robotics Toolbox de Peter Corke en MATLAB primero se modeló el robot mediante parámetros DH. Luego, se realizó un análisis dinámico con el fin de determinar si los actuadores y las relaciones de transmisión seleccionadas eran adecuados para las trayectorias requeridas. Paralelamente, se diseñó y fabricó la estructura mecánica del robot, priorizando ligereza, rigidez y facilidad de manufactura mediante impresión 3D.

En la etapa de control, se implementó una arquitectura distribuida basada en una ESP32 C3 Supermini para el brazo y una Raspberry Pi Zero como autómatas central. La coordinación con el robot móvil se resolvió mediante comunicación MQTT, permitiendo un flujo robusto de mensajes, sincronización por handshake y monitoreo en tiempo real mediante un HMI implementado en Node-RED.

El resultado final es un sistema robótico capaz de ejecutar secuencias de manipulación de forma precisa, repetible y segura, cumpliendo los requisitos funcionales planteados por la cátedra.

3. Objetivos del proyecto

3.1. Objetivos generales

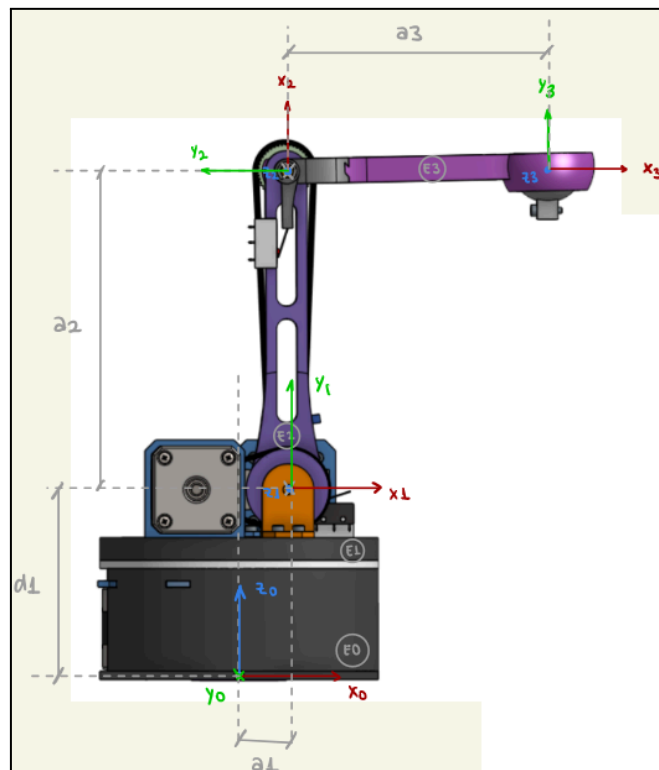
Diseñar e implementar un robot serie antropomórfico de tres grados de libertad capaz de manipular y transportar un vaso con agua de manera precisa, estable y segura, integrando los conceptos de cinemática, dinámica, diseño estructural y control, conforme a los requisitos establecidos por la cátedra de Robótica II.

3.2. Objetivos particulares

- Definir el problema de manipulación y planificar el desarrollo del robot considerando las restricciones dimensionales, materiales y operativas.
- Modelar la cinemática directa e inversa del manipulador para garantizar posicionamiento vertical preciso y trayectorias suaves.
- Diseñar una trayectoria segura que permita trasladar un vaso de 120cc, lleno al 80% de su capacidad, sin derrames, cumpliendo con:
 - al menos 90° de movimiento en la articulación base,
 - al menos 15° de movimiento en las articulaciones restantes.
- Seleccionar e integrar los actuadores y sensores disponibles, validando su capacidad para las exigencias de torque y aceleración.
- Desarrollar el diseño CAD y fabricar la estructura utilizando PLA o MDF, optimizando la relación masa-rigidez.
- Implementar un efector final funcional, compatible con el vaso estandarizado, garantizando estabilidad durante el transporte.
- Asegurar la coordinación con un robot móvil, permitiendo la entrega y recepción del vaso de forma fluida y sin derrames.
- Validar experimentalmente la precisión, repetibilidad y robustez del sistema mediante pruebas controladas.

4. Cinemática

Para el análisis cinemático y en lo posterior para poder llevar a cabo los cálculos se definió el robot a partir de sus parámetros DH. En la siguiente figura se muestra de forma esquemática la definición.



En la siguiente tabla se resumen los parámetros del robot reducido a las 3 articulaciones activas.

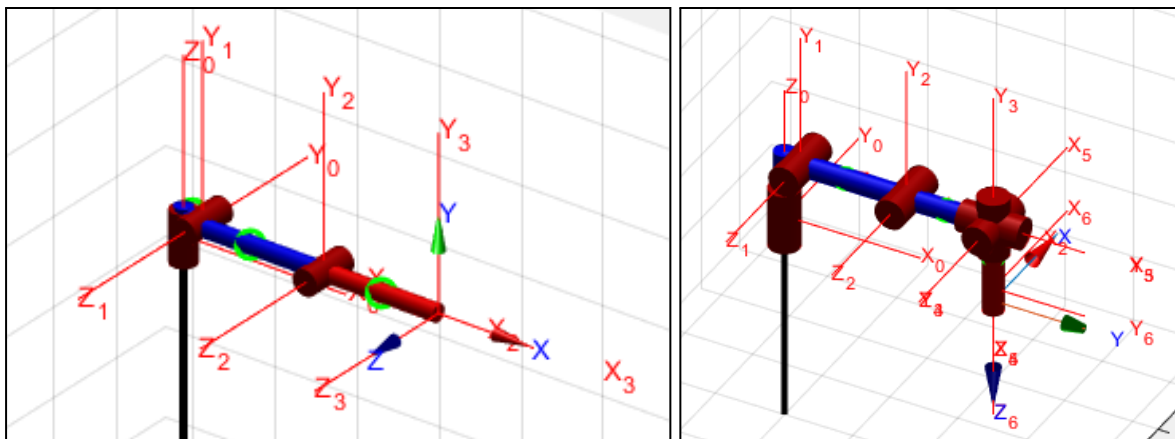
j	q	d [m]	a[m]	α [rad]	offset[rad]
1	q_1	0.08845	0.02315	1.5708	0
2	q_2	0	0.155	0	1.5708
3	q_3	0	0.127	0	-1.5708

Este modelo se completa para representar el robot completo con la articulación esférica pasiva, a la que hay que incorporar los parámetros de longitud del último eslabón pendulante hasta el efector final. Los parámetros DH del modelo de 6 grados de libertad se indican a continuación

j	q	d [m]	a[m]	α [rad]	offset[rad]
1	q_1	0.08845	0.02315	1.5708	0
2	q_2	0	0.155	0	1.5708
3	q_3	0	0.127	0	-1.5708

4	q_4	0	0	1.5708	-1.5708
5	q_5	0	0	-1.5708	-1.5708
6	q_6	0.1	0	0	0

En la siguiente se muestran las representaciones cinemáticas de ambos modelos del robot, a la izquierda con solo las 3 articulaciones actuadas y el derecho con todas las articulaciones presentes en el robot.



5. Dinámica

El análisis dinámico del robot se realizó utilizando el Robotics Toolbox de Peter Corke en MATLAB, aprovechando las funciones de dinámica directa e inversa provistas por la clase SerialLink. El objetivo principal fue verificar si los actuadores y las relaciones de reducción seleccionadas resultaban adecuadas para los esfuerzos requeridos durante la operación.

5.1. Definición de la trayectoria

Se definieron dos configuraciones (inicial y final) dadas en el espacio de las tres articulaciones actuadas. Entre estos puntos se generaron trayectorias por interpolación en el espacio articular utilizando perfiles de velocidad trapezoidal probando con variaciones en velocidad, aceleración y tiempo total procurando trayectorias exigentes. En esta definición de trayectorias no se consideran los últimos 3 grados de libertad RRR de la junta esférica pasiva.

Al llegar a la configuración final, debido a la junta esférica, el último eslabón presenta un movimiento pendular transitorio luego del cual, éste tiende a alinearse verticalmente, permitiendo determinar fácilmente la postura del efector final.

5.2. Modelo dinámico utilizado

Para la realización de los cálculos se importaron en el script de MATLAB los parámetros dinámicos del robot diseñado: masas, tensores de inercia, posiciones de los centros de masa, relaciones de reducción, inercias de los actuadores y coeficientes de fricción estimados.

Se consideró un modelo reducido compuesto por las tres articulaciones actuadas, al que se agregó en el extremo la masa del payload (vaso con agua) y la masa e inercia del último eslabón pendulante, nuevamente sin considerar la junta esférica con los 3 grados de libertad no actuados.

5.3. Cálculo mediante Newton–Euler recursivo

La dinámica inversa se obtuvo a partir de la trayectoria definida y discretizada en el tiempo a partir de posición, velocidad y aceleración articular (q , q_d , q_{dd} respectivamente) mediante la función:

$$\tau = \text{SerialLink.rne}(q, q_d, q_{dd})$$

Esta función implementa el algoritmo Recursive Newton–Euler e incluye los efectos de gravedad, fricción y las contribuciones dinámicas asociadas a las transmisiones.

El resultado fueron los torques necesarios para seguir la trayectoria planificada en el espacio de las articulaciones actuadas.

5.4. Dinámica directa y validación

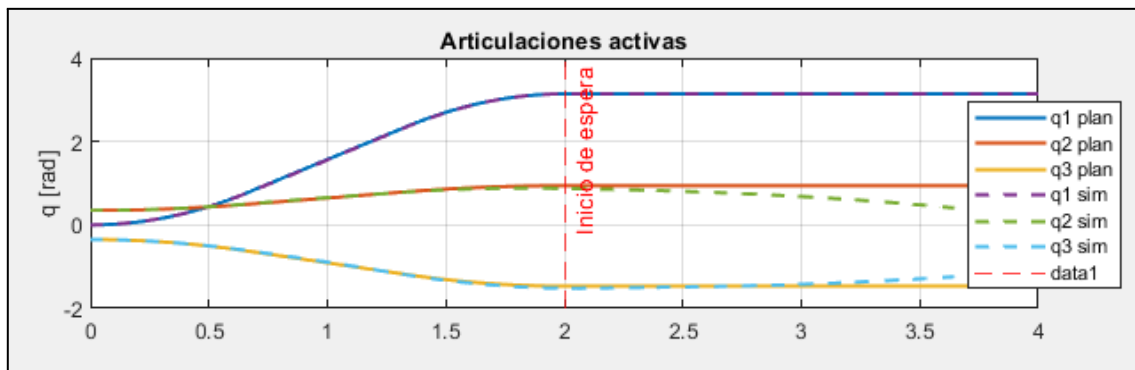
Una vez obtenidos los torques, se aplicaron al **modelo completo de 6 grados de libertad** mediante dinámica directa usando:

$$[t_{sim}, q_t, q_{d_t}] = \text{SerialLink.fdyn}(t, \tau_{fun}, q_0, q_{d_0})$$

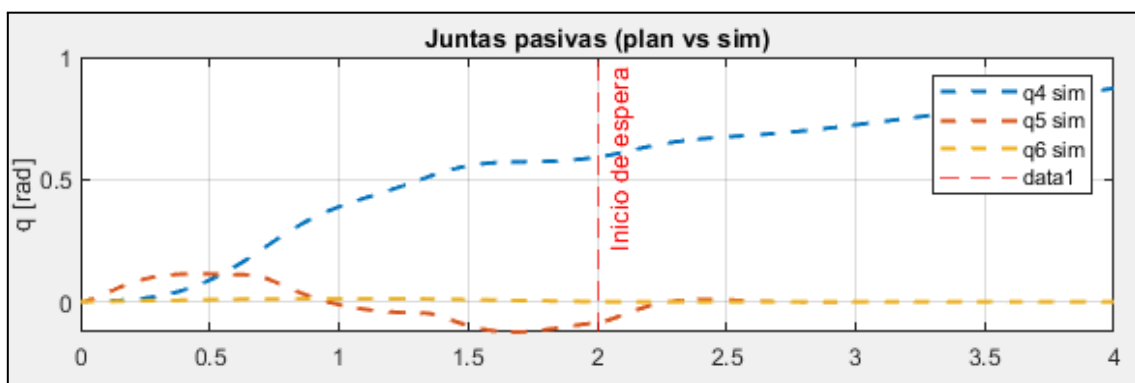
- q_0 : Vector de posiciones articulares inicial
- q_{d_0} : Vector de velocidad articulares inicial
- τ_{fun} : Función que da los torques planificados para la trayectoria
- t : Es el vector de tiempos discretos de la trayectoria.
- t_{sim} : Es el vector de tiempos obtenidos de la función.
- q_t y q_{d_t} : Posiciones y velocidades simuladas.

En esta simulación, las **tres articulaciones actuadas** recibieron los torques calculados por RNEA, mientras que la articulación pasiva se simuló con torque nulo en sus 3 grados de libertad.

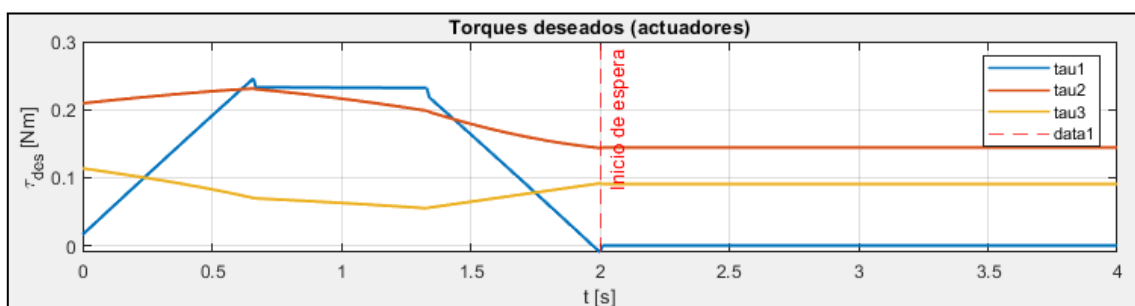
Luego se analizaron las trayectorias resultantes y se compararon con las esperadas para evaluar desviaciones debidas al movimiento de la junta esférica y a las interacciones dinámicas entre los eslabones. En la siguiente figura se compara la trayectoria simulada con la planeada para los primeros 3 grados de libertad, se puede observar que existen diferencias sobre todo hacia el final del movimiento.



Por otro lado, la evolución de los grados de libertad no actuados se muestran en la siguiente figura



En la siguiente gráfica se representan los torques articulares necesarios para la trayectoria planeada.

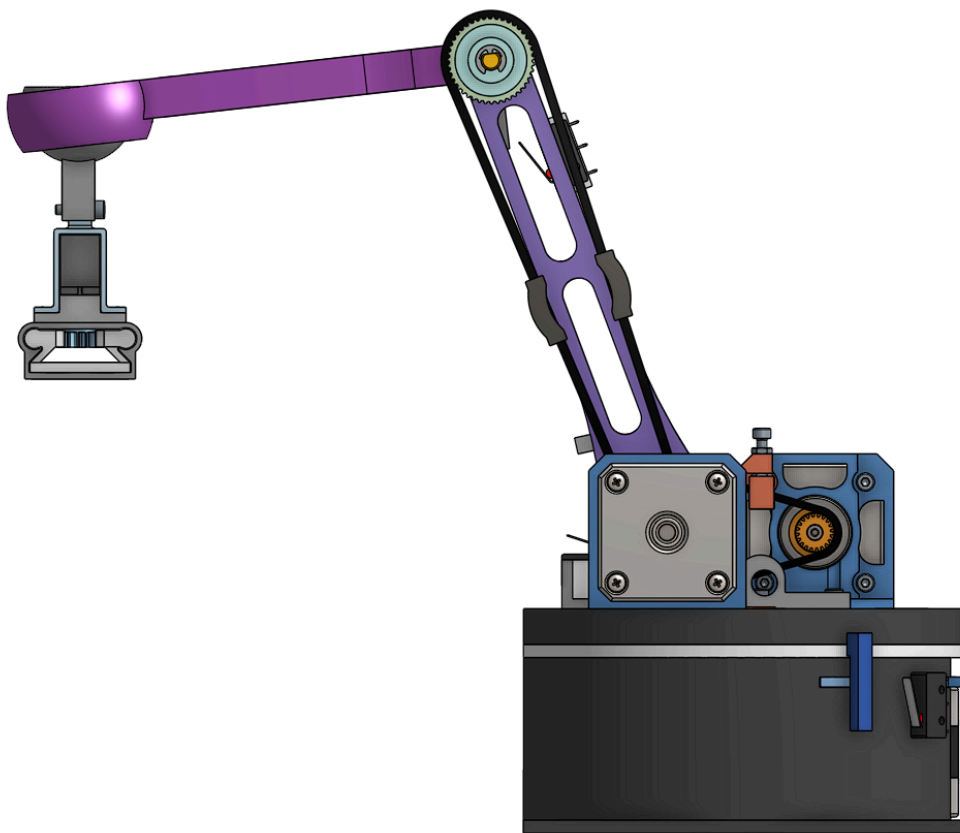


Finalmente, se determinaron los torques articulares máximos requeridos en las 3 articulaciones activas con sus respectivos actuadores y se determinó, mediante relevación en datasheets de actuadores similares (por no disponer los datasheets particulares correspondientes), que los mismos eran menores a los máximos que los mismos podrían entregar.

6. Diseño mecánico

El manipulador de vaso de agua se diseñó como un robot serie de tres grados de libertad (GDL) actuados, configurado para operar en el espacio y realizar las tareas específicas de tomar y trasladar un vaso de 120cc desde una estación designada, hasta el soporte correspondiente en el robot móvil. Esta configuración se seleccionó buscando un equilibrio entre simplicidad mecánica, facilidad de control y la capacidad funcional requerida.

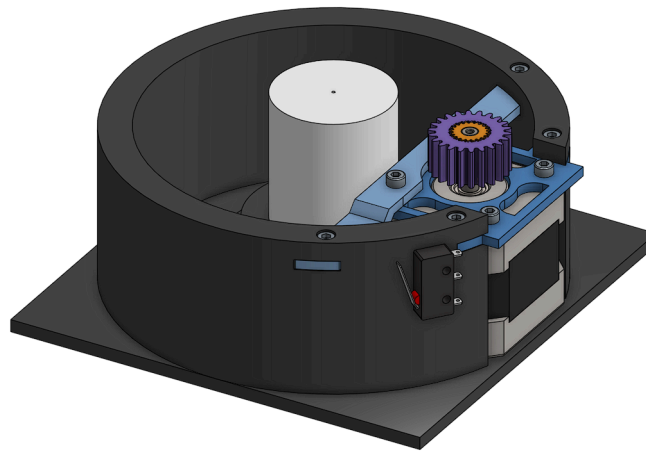
Este diseño, realizado mediante la herramienta colaborativa de CAD *Onshape*, se compone de una base fija que soporta tres eslabones articulados por juntas rotacionales (una configuración RRR), confiriéndole su capacidad de movimiento.



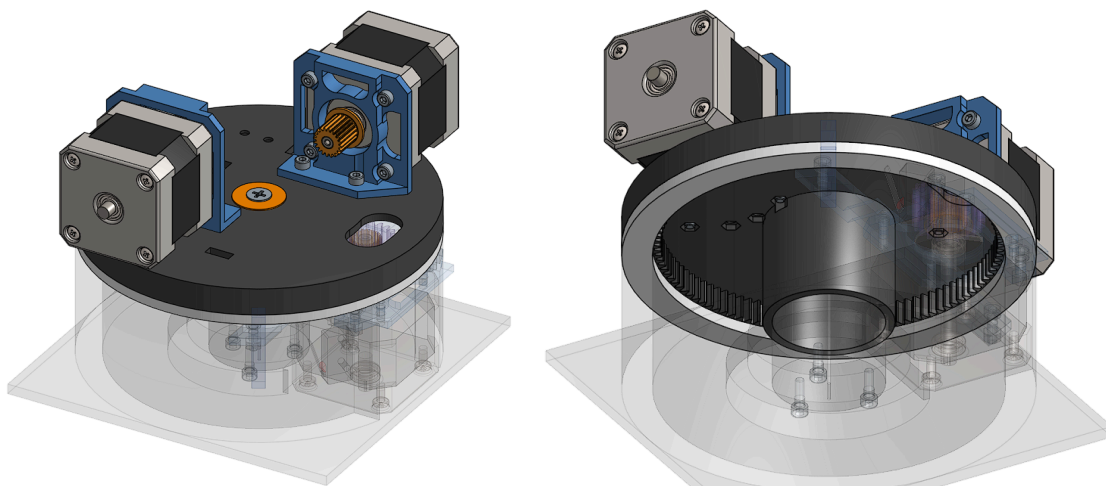
6.1. Descripción de eslabones

El diseño estructural del manipulador se compone de tres eslabones actuados principales (E1, E2 y E3) que definen su alcance y movilidad en el espacio de trabajo:

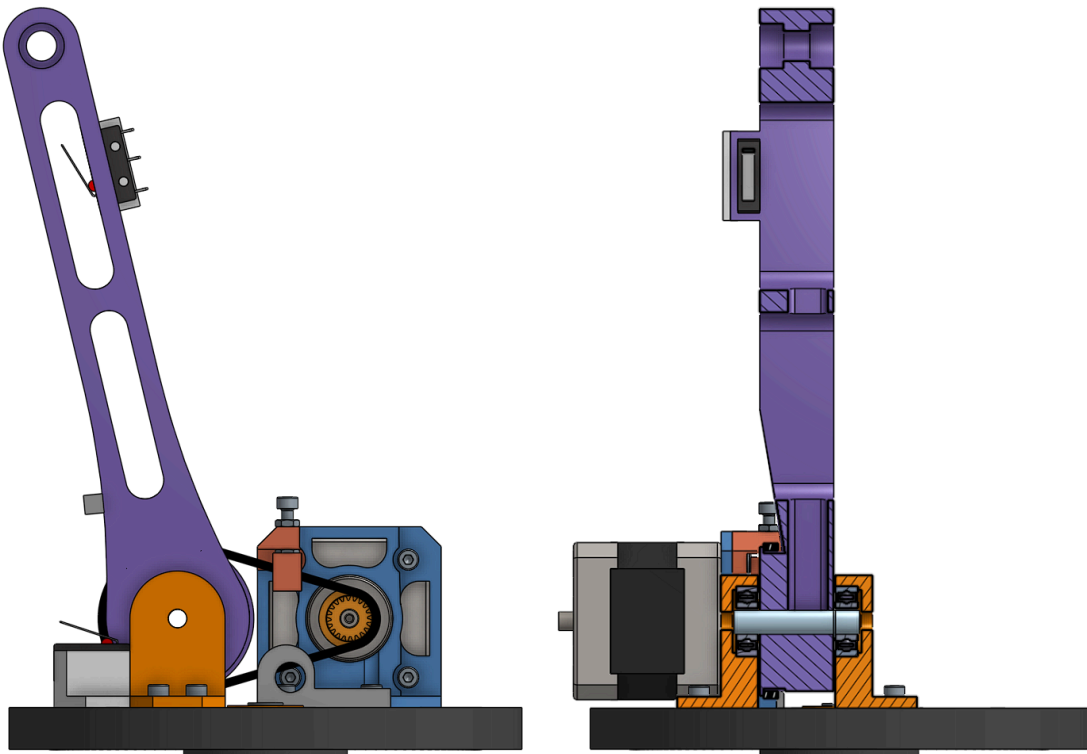
Base fija (E0): Esta estructura no móvil sirve como anclaje del manipulador al entorno de trabajo. Incluye un eje central de Acrílico Polimetilmetacrilato (APM) de 35 mm de diámetro, sobre el cual rotará el siguiente eslabón (uniéndose a la primera articulación, J1). También fue diseñada con ranuras y medidas específicas para el anclaje del motor. Su función es soportar la carga estática y dinámica generada por el movimiento del brazo.



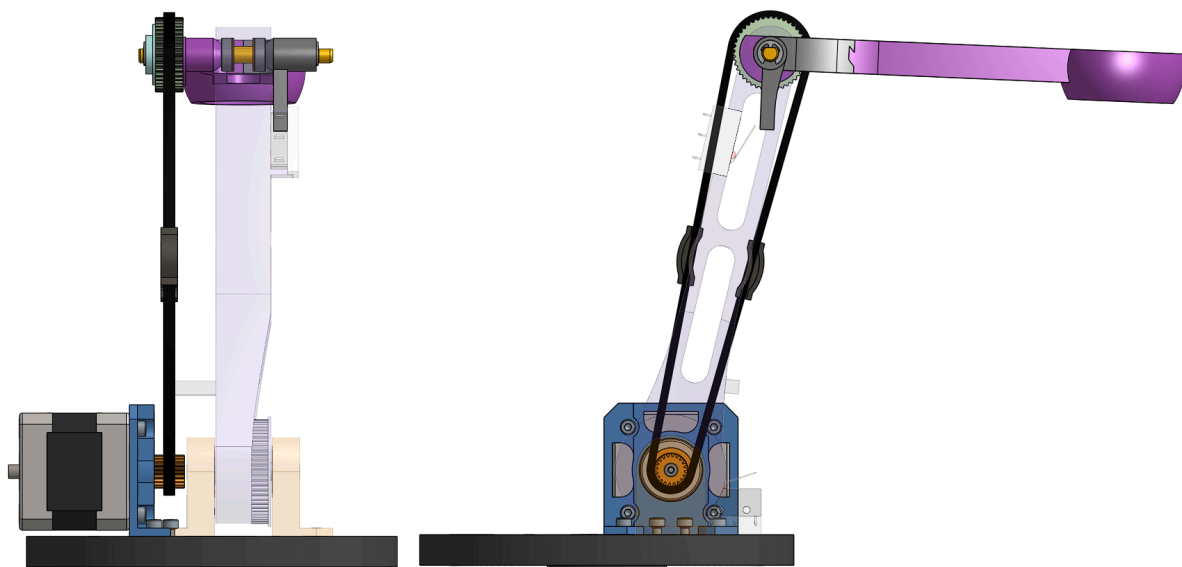
La **Base Móvil (E1)** se conecta a la base fija (E0) para formar la primera articulación rotacional (J1). Su movimiento, generado por un engranaje de dentado interno y el adaptador en el eje motor, permite la rotación de todo el conjunto superior (movimiento de barrido principal) alrededor del eje vertical. La interfaz de movimiento utiliza un eje y una junta de APM para minimizar la fricción. Se incluye una arandela de seguridad como tope para prevenir el posible desprendimiento del eslabón causado por momento flector que el brazo cargado y en extensión pudiese ocasionar.



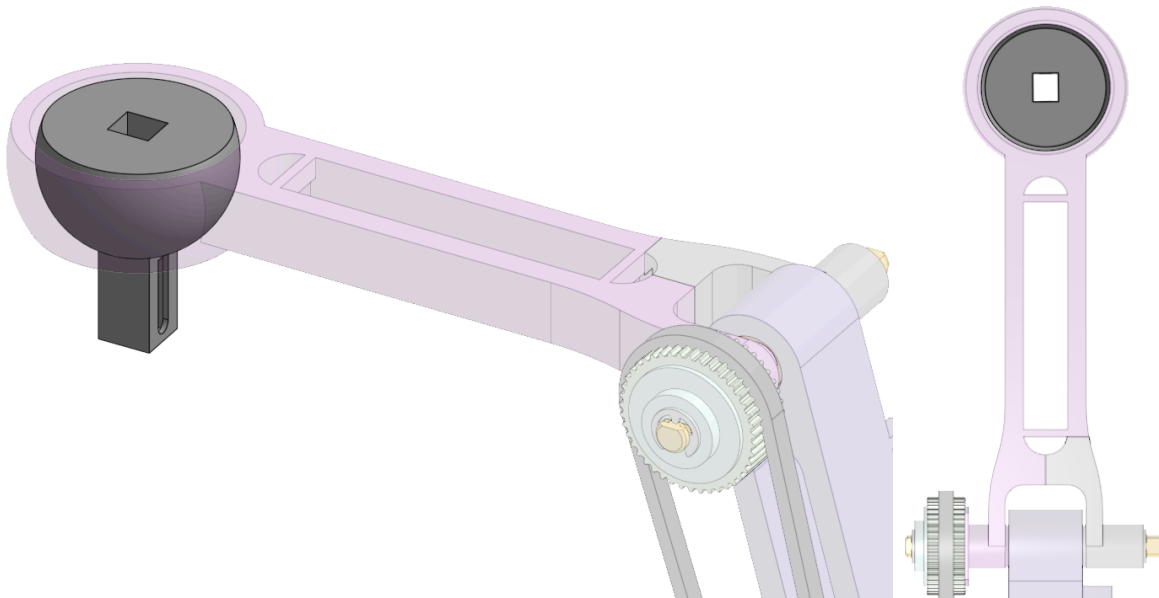
Eslabón 2 (Brazo): El Motor 2, montado sobre el Eslabón 1 (Base Móvil), impulsa el Eslabón 2 a través de una correa dentada, constituyendo la articulación 2 (J2). Su propósito esencial es la extensión y retracción del brazo en un plano vertical. Se incluye un corte detallado para visualizar la estructura de los soportes del eslabón, destacando los rodamientos sobre los cuales se encuentra montado el eje.



Eslabón 3 (Antebrazo): Se conecta al Eslabón 2 a través de la Articulación 3 (J3). Su función principal es proporcionar extensión horizontal al robot y servir de soporte para el efector final. El diseño incorpora un sistema de transmisión de potencia por correa, conectando la polea en la base móvil del eje motor con una polea superior fija al eje articular. A pesar de que el motor de este eslabón se encuentra en la base móvil (E1), su eje coincide con el de la Articulación 2. Esta configuración permite que, al mover J2, no sea necesario accionar J3. La estructura ha sido optimizada para minimizar el peso, manteniendo la rigidez necesaria para las operaciones de manipulación.

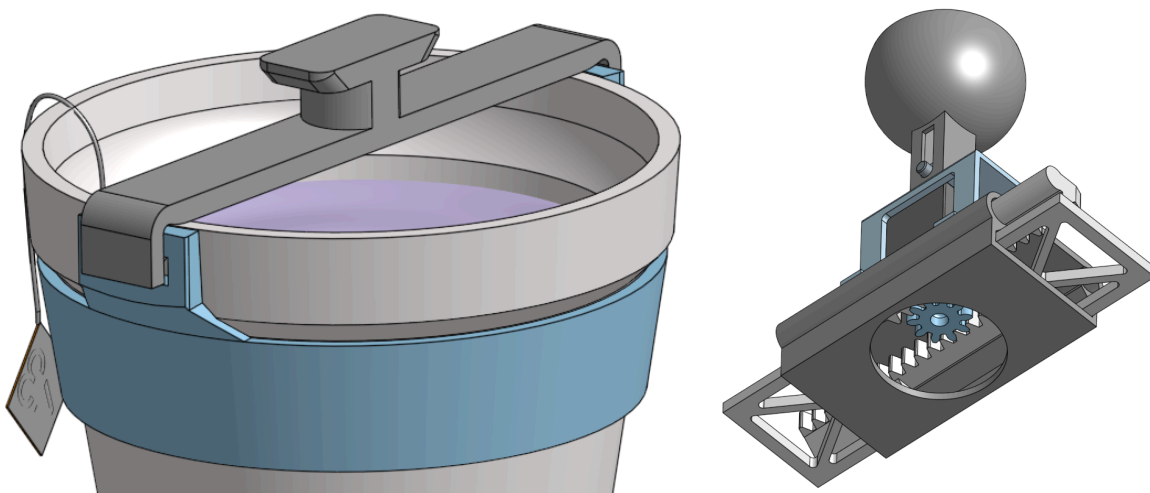


Eslabón 4 (muñeca): Seguido del eslabón 3, se encuentra una articulación esférica que proporciona 3 grados de libertad rotacional de manera pasiva. No está conectada a ningún actuador que controle su movimiento, por lo cual su orientación es libre y está sujeta al efecto gravitacional. Esta articulación es la base de montaje para el gripper efector final.

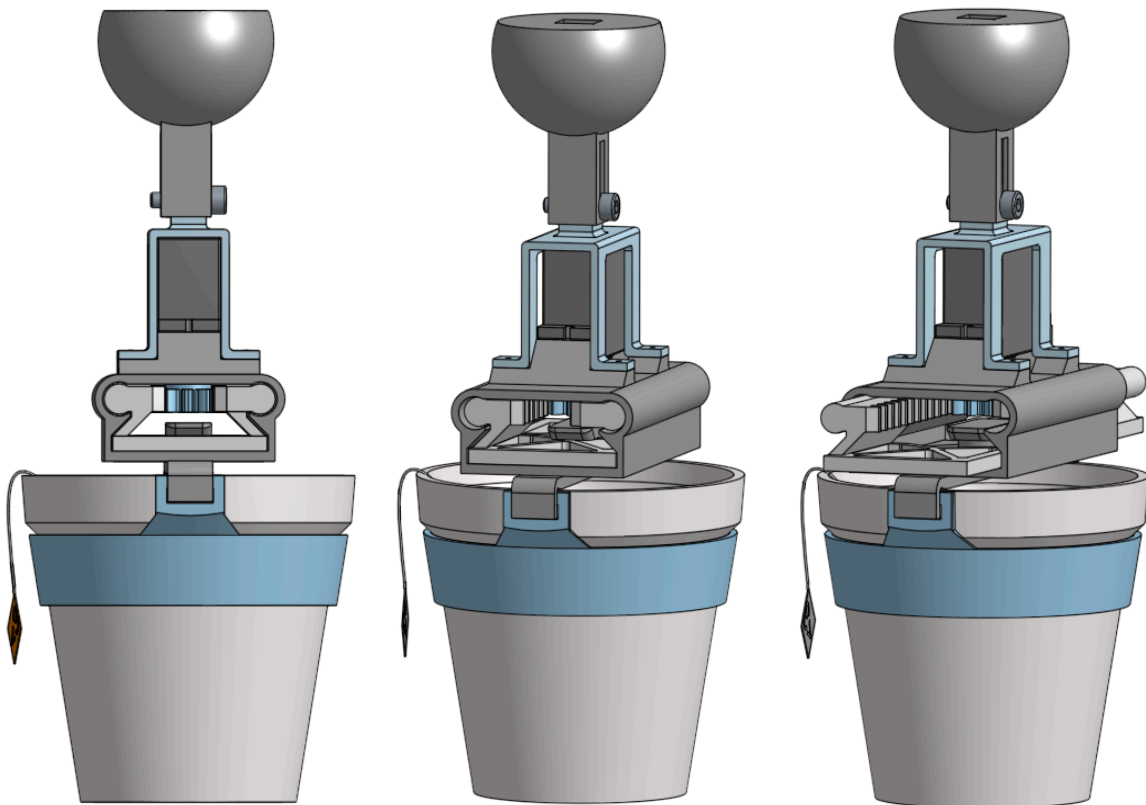


6.2. Efecto final

Para la manipulación del vaso por el brazo robótico, se diseñó una estructura de soporte dedicada. Esta estructura cumple una doble función: asegurar la estabilidad del vaso y servir como punto de anclaje para el gripper.



El mecanismo de agarre consiste en un sistema de engranaje y cremalleras acoplado a un servomotor, que permite la apertura y cierre de la pinza. Este diseño asegura que el brazo robótico manipule el conjunto (estructura + vaso) como una sola pieza, garantizando la sujeción sobre la estructura robusta para proteger el vaso y su contenido.



6.3. Materiales empleados en la construcción

La mayor parte del manipulador se fabricó utilizando PLA impreso en 3D. Este material se seleccionó por su excelente relación entre facilidad de prototipado, ligereza y resistencia mecánica (relativa al uso), siendo ideal para los eslabones, soportes de motores, poleas y el efector final, contribuyendo además a la minimización de la inercia del sistema.

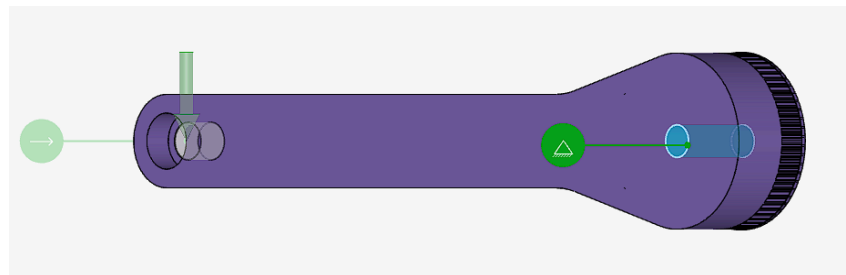
Para las zonas críticas que requieren baja fricción y alta resistencia al desgaste, como el eje principal de la Base Fija (E0) y la junta de apoyo en la articulación J1, se empleó Acrílico Polimetilmetacrilato (APM). Finalmente, la integridad estructural de las uniones se aseguró mediante el uso de tornillos y tuercas metálicas de acero.

Para facilitar el movimiento en las articulaciones y minimizar la fricción, se integraron rodamientos de bolas, también metálicos. Los ejes de las articulaciones se obtuvieron mediante la reutilización de ejes metálicos extraídos de impresoras y equipos electrónicos desmantelados.

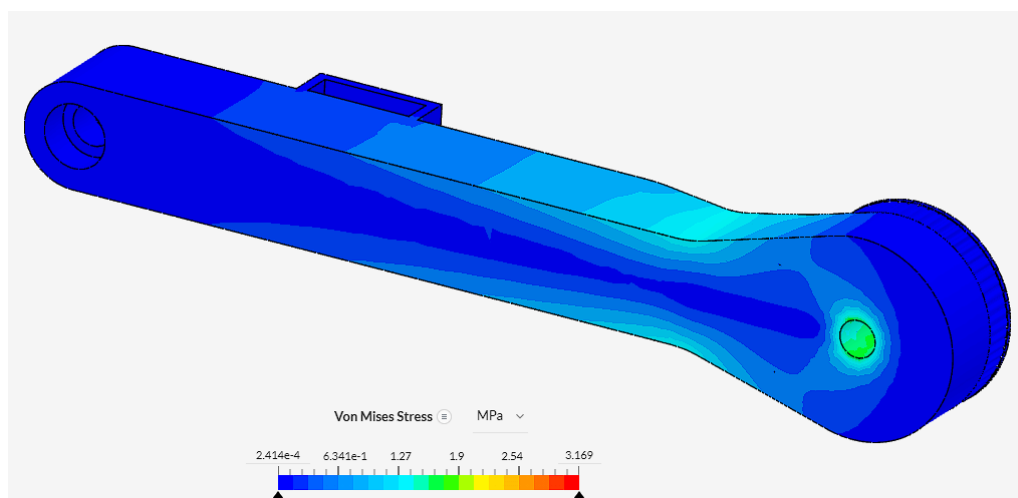
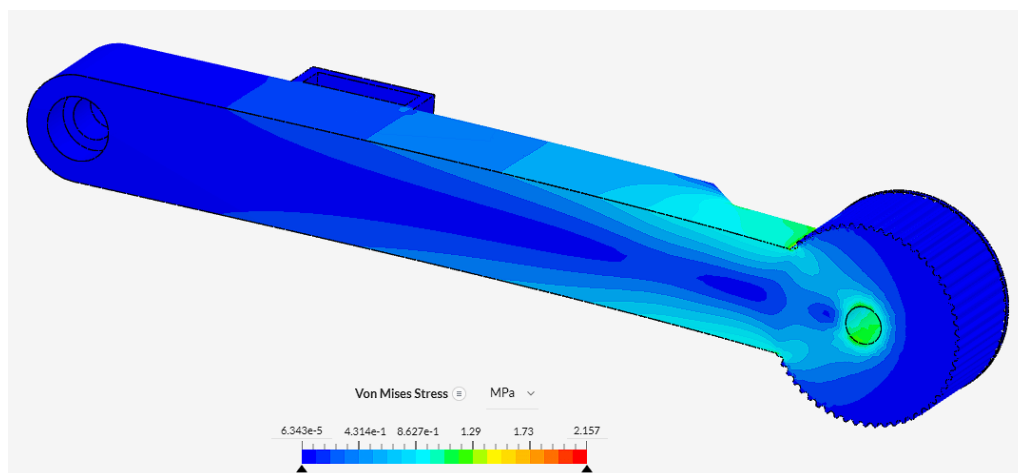
6.4. Verificación de esfuerzos y optimización de los eslabones

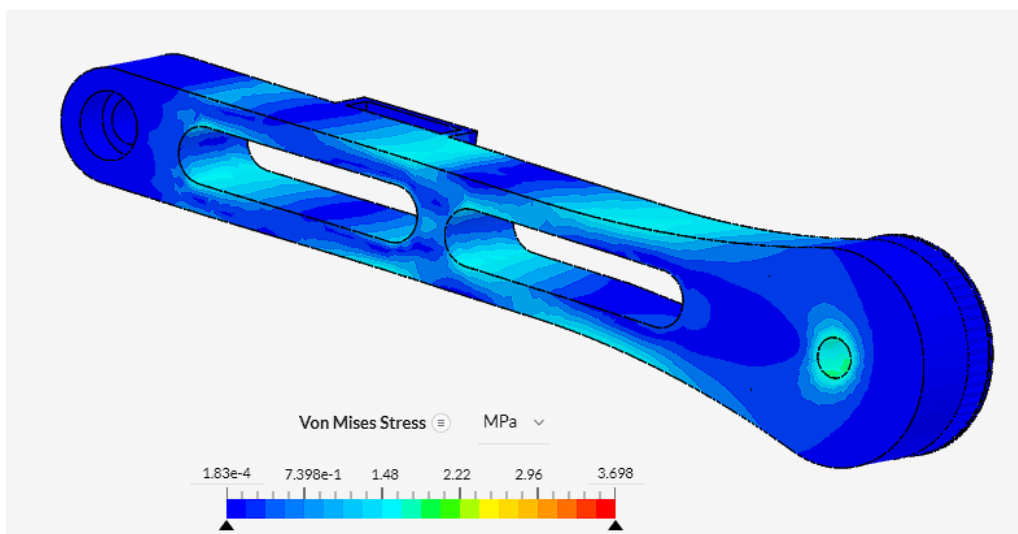
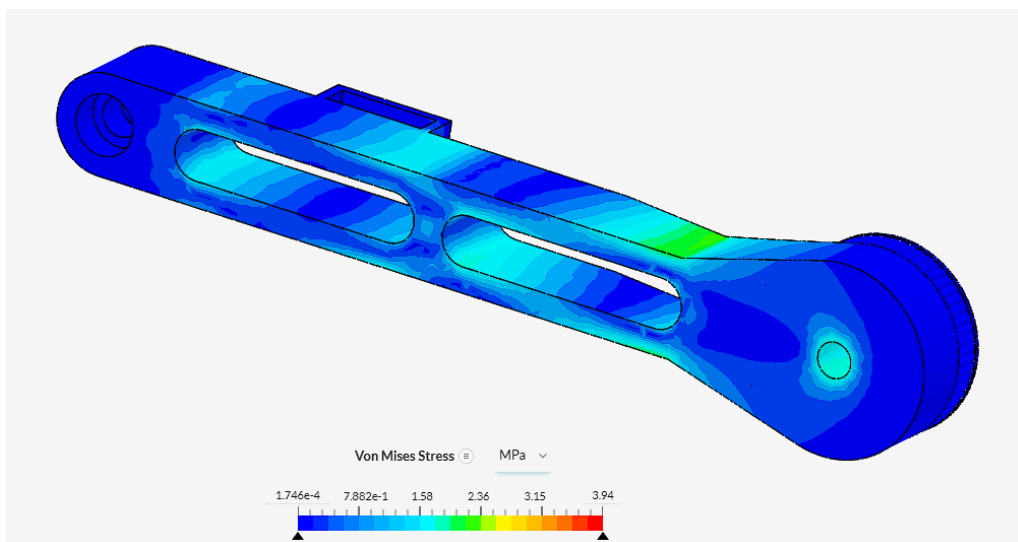
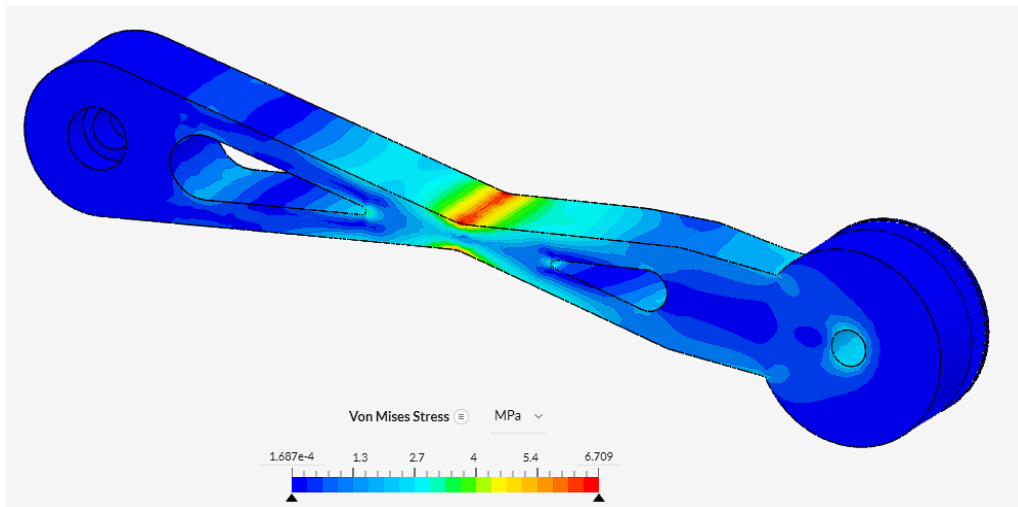
El procedimiento detallado a continuación se llevó a cabo para asegurar la integridad estructural y optimizar el rendimiento de los componentes del brazo robótico. La optimización de los eslabones diseñados se realizó utilizando la herramienta *Simscale*, la cual facilitó el análisis de los modelos mediante su integración con *Onshape* para la exportación directa de los mismos.

Con el fin de lograr un diseño que reduzca la cantidad de material, pero no por esto perder rigidez estructural, los diferentes modelos desarrollados se sometieron a las mismas condiciones de borde y carga. Para la simulación, se propuso la configuración del brazo completamente extendido. Esto implicó simular un empotramiento en el eje de soporte del eslabón y aplicar una carga en el extremo final del brazo. Esta carga fue calculada en función del momento flector que la carga real originaría al aplicarla en el extremo del siguiente eslabón (E3). Considerando una unión completamente rígida en la interfaz E2-E3 y las distancias correspondientes, la carga en la punta del E2 resulta ser de 800 g. También se debe tener en cuenta que la gravedad afecta en el mismo sentido que la fuerza aplicada.

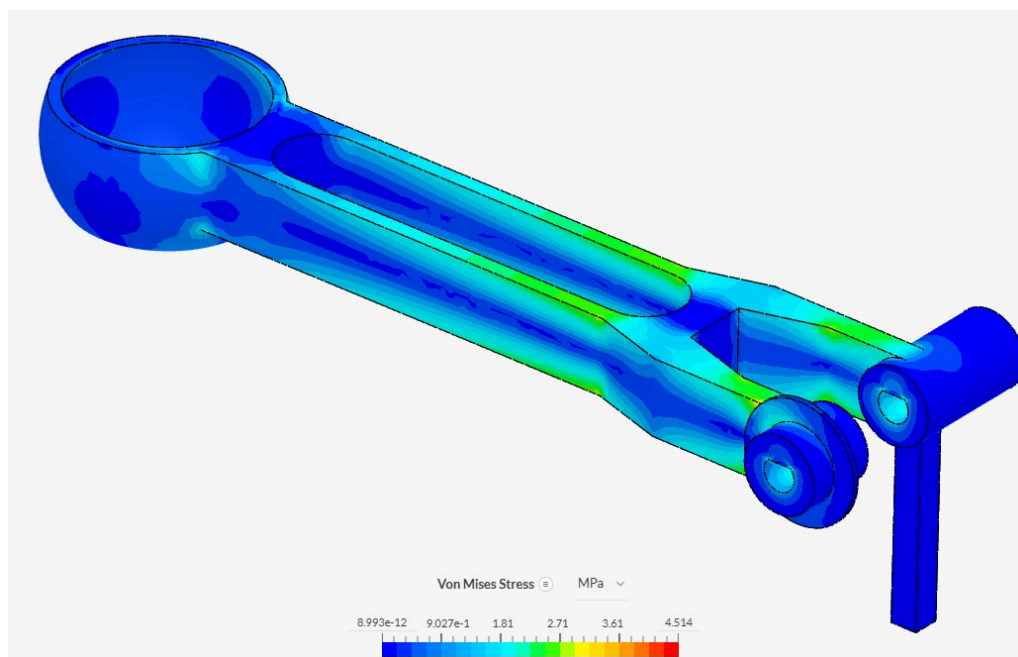
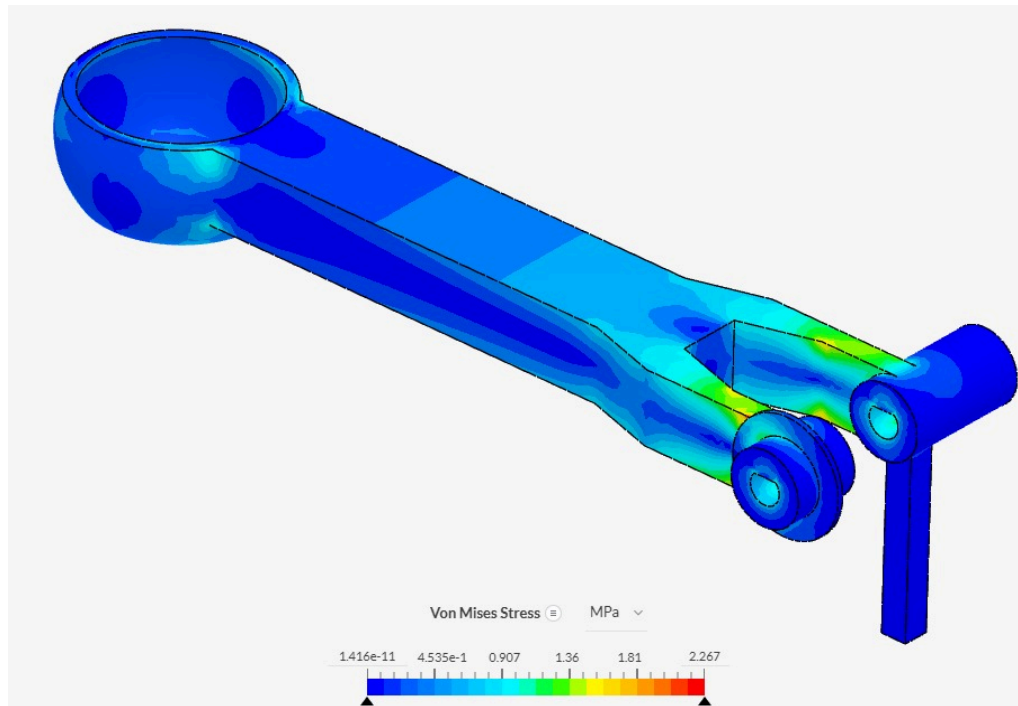


Importado el primer diseño y aplicadas las condiciones necesarias, se procedió con la simulación y optimización de las geometrías, logrando la siguiente evolución del eslabón 2.





Para el eslabón 3 se realizó el mismo procedimiento, pero ahora considerando una carga de 500 g en su extremo final, donde se montará el gripper. Nuevamente se consideró la condición completamente extendida, lo cual implica la carga actuando de manera vertical hacia abajo, también la gravedad y un empotramiento al eslabón anterior, ubicado en su eje.



Una vez finalizadas las simulaciones y la etapa de optimización, las geometrías de los eslabones **E2 (Brazo)** y **E3 (Antebrazo)** se consideraron definitivas. **Cabe destacar que las simulaciones se realizaron asumiendo una densidad uniforme del material (PLA), una simplificación que no se corresponde con la naturaleza heterogénea de las**

piezas impresas en 3D. No obstante, en la práctica, las piezas se imprimieron orientadas de manera que se logre la máxima resistencia mecánica posible, lo que validó el diseño. Las pruebas de tensión y deformación demostraron que, en la configuración de máxima exigencia (brazo completamente extendido y cargado con el efector final y el vaso), las tensiones máximas alcanzadas son sumamente razonables y se encuentran muy por debajo del límite elástico del material.

En concreto, los picos de tensión calculados fueron de aproximadamente el 10% (3-5 MPa) de la tensión máxima admisible por el PLA (53-55 MPa) al límite elástico, lo que garantiza un amplio margen de seguridad y una rigidez adecuada para la manipulación. Este proceso de optimización, que consistió en el aligeramiento de las piezas sin comprometer la resistencia estructural, resultó exitoso, permitiendo reducir la masa total del manipulador, las horas dedicadas a su conformación y, consecuentemente, la disminución de las inercias y los requerimientos de torque dinámico sobre los actuadores.

7. Selección de actuadores

Los actuadores utilizados son motores paso a paso para las 3 articulaciones activas y un servomotor para el gripper. Estos elementos se obtuvieron reciclados de impresoras mientras que el servomotor para el gripper estaba disponible.

A continuación se detallan las especificaciones relevantes de los actuadores disponibles:

Motor	Código	Ángulo de paso	Corriente	Torque estimado
Motor art. 1	STP 43D2052	1.8°/paso	1.1 A	0.32 Nm
Motor art. 2	STP 43D2052	1.8°/paso	1.1 A	0.32 Nm
Motor art. 3	STP 43D2051	1.8°/paso	0.9 A	0.32 Nm

Gripper	SG92R	Rango 0 a 180 °	Torque: 2.5kg /cm
---------	-------	-----------------	-------------------

El sistema de transmisión que se escogió fue el de correas y poleas. Las correas fueron recicladas también de impresoras mientras que las poleas fueron impresas de acuerdo a los parámetros de las correas. Las relaciones de transmisión se determinaron a partir del análisis dinámico de modo que las capacidades de los motores stepper sean suficientes.

Articulación	Relación de transmisión
Art. 1	5
Art. 2	3
Art. 3	2,15

8. Programación

8.1. Controlador local

El sistema de control fue implementado en una placa ESP32 C3 Supermini, y consiste en un controlador local a lazo abierto, monoarticular, basado en la ejecución directa de posiciones objetivo expresadas en pasos. Cada articulación recibe consignas independientes, y la sincronización entre ejes se logra mediante interpolación lineal coordinada en espacio articular, con perfiles de velocidad trapezoidales provistos por las librerías de movimiento utilizadas.

El movimiento se define a partir de puntos característicos cargados manualmente (posición de homing, puntos intermedios de la trayectoria y posición final). A partir de estos puntos, se generan trayectorias que respetan límites máximos de velocidad y aceleración parametrizados individualmente en cada motor.

El enfoque de control se basa en:

- Lazo abierto: las posiciones se envían directamente en pasos sin realimentación de posición (aunque el sistema dispone de finales de carrera para homing).
- Control monoarticular: cada motor gobierna una sola articulación y ejecuta movimientos absolutos o relativos según consignas.
- Interpolación coordinada: el cálculo de los tiempos se realiza para que todos los ejes alcancen sus posiciones objetivo de manera simultánea, evitando descoordinaciones en el transporte del vaso.
- Perfiles trapezoidales: aceleración - velocidad constante - desaceleración, provistos internamente por FastAccelStepper y AccelStepper.

Arquitectura del software

El código se implementó con un enfoque de programación orientada a objetos, encapsulando la lógica de cada actuador en clases específicas. Esto permite un código modular, fácil de extender y de mantener.

Las clases desarrolladas fueron:

Clase miStepper (Hombro y Codo)

Basada en la librería FastAccelStepper, optimizada para alto rendimiento en la ESP32-C3, especialmente para el control preciso de motores paso a paso.

Funcionalidades principales:

- Inicialización del motor y asociación con el engine global.
- Configuración de velocidad y aceleración máximas.
- Ejecución de movimientos absolutos (goToPos) y relativos (move).
- Conversión entre posición lógica (DH, siempre positiva) y posición física (según dirSign).
- Detección de movimiento en curso (isBusy).
- Rutina de homing con final de carrera, incluida la lógica de pre-backoff, búsqueda y posicionamiento cero.
- Forzado de parada con reposicionamiento seguro (stopMove).

Esta clase encapsula todo el comportamiento de las articulaciones “superiores”, donde la velocidad no es tan crítica como la precisión del stepping y el tiempo de ejecución no está limitado por interrupciones del ESP32-C3.

Clase miStepperBase (Base)

Implementa el movimiento mediante la librería AccelStepper, seleccionada porque permite realizar stepping por software sin bloqueo y funciona correctamente en paralelo con FastAccelStepper.

Principales características:

- Control del motor principal de la base, donde las velocidades requeridas son menores y el stepping no necesita la precisión temporal estricta de FastAccelStepper.
- Soporte para movimientos absolutos y relativos.
- Rutina de homing con búsqueda del final de carrera, retroceso controlado y posicionamiento cero.
- Manejo explícito de la señal ENA (enable) del driver, que AccelStepper no administra por defecto.
- Método run() indispensable para actualizar el stepping en cada iteración del ciclo.

La coexistencia de ambas librerías fue necesaria debido a las limitaciones de la ESP32-C3 Supermini, que no permite asignar todos los ejes simultáneamente a FastAccelStepper (pines disponibles, timers internos y uso de interrupciones).

Clase miServo (efector final)

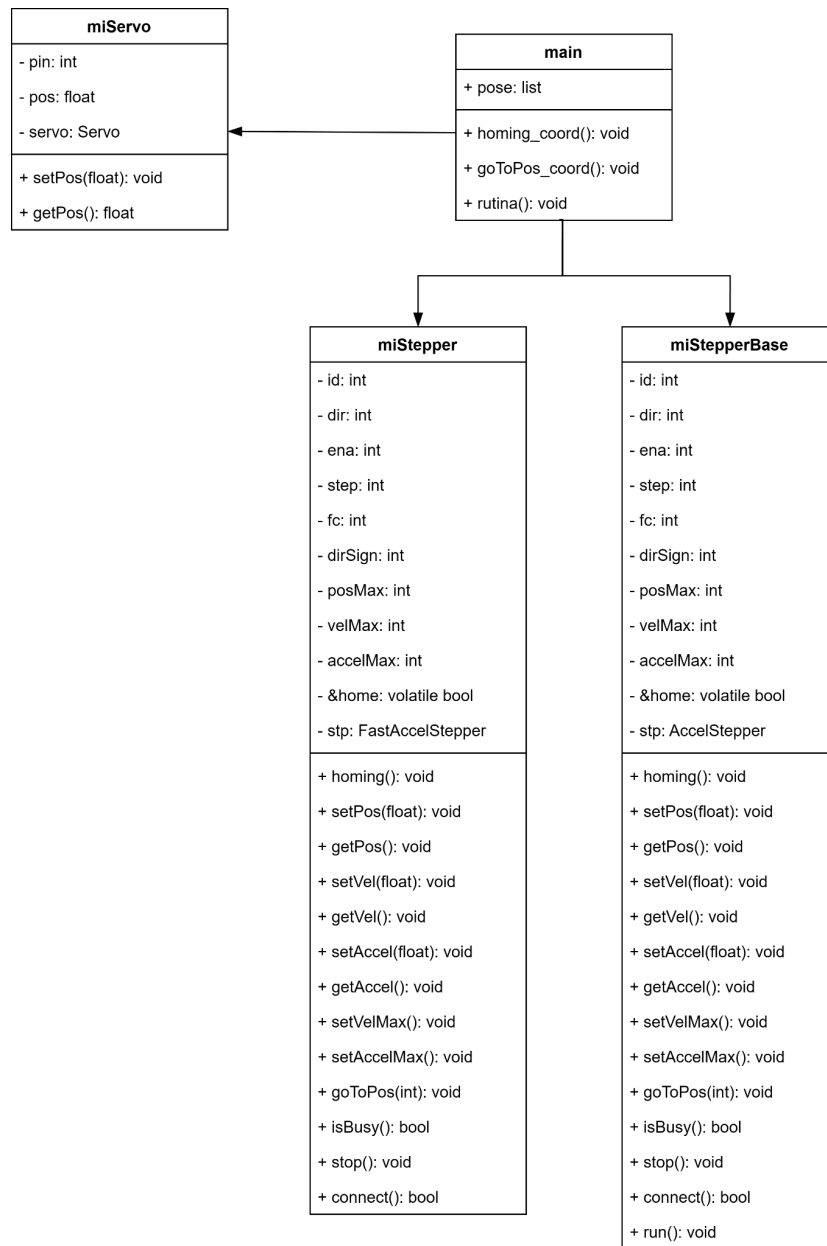
Basada en ESP32Servo, se implementa un objeto simple para controlar el servo encargado de la pinza de sujeción del vaso.

Funciones principales:

- Inicialización del servo a 50 Hz y configuración del rango de pulsos.
- Método setPos() que limita la posición entre 15° y 150° (ángulos determinados experimentalmente para apertura y cierre del efector).
- Lectura del último valor seteado mediante getPos().

El módulo principal (main) crea instancias de estas clases y se encarga de la lógica de alto nivel, incluyendo homing coordinado, ejecución de trayectorias y manejo de comandos por puerto serie.

Lo anterior se resume en el diagrama de clases que se muestra a continuación:



Estrategia de movimiento coordinado

Para asegurar que los tres ejes se muevan y detengan simultáneamente, se implementó una función de Interpolación Lineal Coordinada en el espacio articular, que funciona de la siguiente manera:

1. Cálculo de distancias: Se calcula la distancia lógica que debe recorrer cada articulación (diferencia de pasos entre posición deseada y posición actual, en valor absoluto).
2. Determinación de tiempo máximo: Se calcula el tiempo de viaje que requeriría cada articulación para completar su movimiento a su velocidad máxima y aceleración máxima configuradas. El mayor de estos tres tiempos define el tiempo que durará el movimiento coordinado.

3. Sincronización de velocidades: Se recalcula la velocidad de cada eje de modo que cada articulación complete su distancia exactamente en el tiempo máximo. La aceleración se mantiene como la máxima configurada.
4. Ejecución: Las nuevas velocidades sincronizadas se establecen en cada motor, y se les comanda el movimiento absoluto. Esto garantiza que los ejes más lentos o con mayor distancia a recorrer dicten el tiempo total, y el robot se mueva en línea recta en el espacio articular.
5. Espera de finalización: El programa principal espera hasta que los tres ejes hayan finalizado su movimiento antes de continuar con un nuevo comando.

Para cada movimiento, las librerías implementan un perfil de velocidad con forma trapezoidal. Esto asegura una aceleración suave al inicio del movimiento, una fase de velocidad constante (si la distancia lo permite), y una desaceleración suave al final, lo cual es crucial para la manipulación de objetos inestables como el vaso de agua.

Rutina de homing

Se implementó una rutina de homing coordinado que utiliza los finales de carrera asociados a cada articulación como punto de referencia cero. La rutina se ejecuta en un orden específico (J3, J2, J1) para evitar colisiones:

1. Homing del Codo (J3).
2. Movimiento de seguridad J3: Se mueve a una posición segura antes de mover el codo.
3. Homing del Hombro (J2).
4. Movimiento de seguridad J2 y J3: Se mueven a una posición segura antes de mover la base.
5. Homing de la Base (J1).

La detección del FC se realiza mediante interrupciones (ISR) configuradas con FALLING edge, actualizando flags booleanos volátiles que son chequeados dentro de la lógica de homing de las clases `miStepperBase` y `miStepper`.

8.2. Autómata para sincronización de actividades de robot serie y robot móvil

Para realizar la sincronización entre el brazo robot y el robot móvil, se implementó una arquitectura de control centralizada. El rol de "Autómata" o cerebro central se delegó a una Raspberry Pi Zero W v1.1, la cual gestiona toda la lógica del proceso.

La comunicación entre el autómata y los esclavos (los ESP32 de los robots) se realiza a través de una red local WiFi mediante el protocolo **MQTT**. Se instaló un broker **Mosquitto** en la Raspberry Pi para gestionar la mensajería. Se eligió MQTT por su modelo de publicación/suscripción, que es extremadamente ligero (ideal para IoT) y permite un **desacoplamiento total**: los robots no se comunican entre sí, sino que reportan su estado al broker. Esto también añade flexibilidad, ya que permite una fácil depuración y la adición de múltiples "clientes" (como el HMI) escuchando los mismos tópicos de estado.

Para la creación del autómata fue útil Node-RED, que es una herramienta de programación visual robusta, de fácil aprendizaje utilizada también en IoT. El autómata se comunica con los robots (Brazo y Móvil) a través del broker MQTT (Mosquitto), suscribiéndose a sus tópicos de estado (/status) y publicando en sus tópicos de control (/control).

La lógica principal del autómata se centra en el cumplimiento de la siguiente secuencia.

- **Estado de Reposo (IDLE):** El autómata monitorea el tópico *MOVIL/estado*. La rutina solo puede iniciar si el robot móvil está en posición (publicando C) y se recibe el comando *START_RUTINA* desde el HMI.
- **Secuencia de Entrega:** Al recibir la señal de inicio, el autómata carga la trayectoria *entrega.json* desde la memoria de la Raspberry Pi. Inicia un bucle de handshake:
 - Envía la Pose N (un objeto con 10 parámetros) al tópico del brazo. El ESP32 la recibe, calcula la interpolación y ejecuta el movimiento. Al finalizar, el ESP32 publica *COMPLETADO* en el tópico *BRAZO/estado*.
 - El autómata recibe este mensaje y recién entonces envía la Pose N+1.
- **Transporte:** Al completarse la última pose de entrega (lo que implica que el vaso fue soltado), el autómata transiciona al estado *TRANSPORTANDO* y envía el comando correspondiente al inicio de transporte (el comando N) al tópico de control del robot móvil.
- **Regreso del Móvil:** El autómata espera a que el móvil complete su ciclo y regrese. Esto se confirma cuando el móvil publica nuevamente C en el tópico *MOVIL/estado*.
- **Secuencia de Devolución:** Al confirmar el regreso del móvil, el autómata carga la segunda trayectoria, *devolucion.json*, y la ejecuta usando exactamente el mismo método de "handshake" (Pose-OK-Pose) con el brazo robot.
- **Fin de Ciclo:** Al recibir el último *COMPLETADO* de la secuencia de devolución, el sistema vuelve al estado IDLE, quedando listo para iniciar un nuevo ciclo.

La interfaz HMI fue también creada con NodeRED. Es una interfaz simple, ideal para un prototipo rápido, pero limitada en el sentido de que es un poco rígida a la hora de cambiar el diseño en la pantalla. Dicha interfaz presentaba las siguientes funcionalidades:

- **Control Manual:** La interfaz permite el "jogging" (movimiento manual) de cada articulación y el ajuste de parámetros (Velocidad, Aceleración).
- **Cinemática (IK/FK):** Se implementaron funciones en JavaScript (dentro de Node-RED) para traducir coordenadas cartesianas (X,Y,Z) a ángulos articulares (Cinemática Inversa) y viceversa (Cinemática Directa), permitiendo un control más intuitivo.
- **"Teach Pendant" (Programación de Poses):**
 - El HMI permite al usuario mover el robot a una posición deseada (usando sliders) y "Guardar Posición".
 - Estas poses (que incluyen los 10 parámetros) se almacenan en una variable de memoria.
 - El HMI permite guardar estas secuencias de memoria en archivos **JSON** persistentes en la Raspberry Pi (*entrega.json*, *devolucion.json*).
 - El autómata puede **cargar** estos archivos JSON para ejecutar rutinas complejas.
- **Monitor de estado:** La interfaz muestra el estado actualizado de ambos robots (si se están moviendo, si están parados, si el móvil está cargado, etc).
- **Control de variables del robot móvil:** Para el robot móvil, es posible cambiar las constantes de PID, así como decirle manualmente que se ponga en estado de carga, pararlo, sacarlo del lugar de carga, etc. Es posible monitorear sus valores de velocidad y de corriente, ya que el robot envía periódicamente información de estas variables gracias a sus sensores.

Para la robustez de este sistema, se implementó:

- **LWT (Last Will and Testament):** Se configuró el ESP32 para que el broker MQTT publique automáticamente un mensaje de *OFFLINE* si el robot se desconecta, permitiendo al HMI mostrar un estado de conexión real.
- **Sincronización de Estado:** Al reconectarse, el ESP32 publica inmediatamente su posición actual, actualizando los sliders del HMI para evitar una desincronización entre el robot físico y la interfaz digital.
- **Parada de Emergencia:** Se implementó un tópico de *STOP* que, al ser recibido, activa una bandera *emergencia_activa* en el ESP32, interrumpiendo maniobras críticas, como el homing o una trayectoria, al instante.

Una desventaja obvia de esta solución, es que la ESP32 del brazo no conoce de trayectorias, pues sólo sabe seguir poses que el autómatas le envía, por lo que si la red se cae en medio de una trayectoria, el robot quedará inútil, esperando a que se envíe la próxima pose. Una forma de solucionar esto es agregando una funcionalidad más para guardar archivos de trayectoria en la propia ESP32, utilizando el sistema de archivos de la memoria flash interna (**SPIFFS** o **LittleFS**) o bien un módulo de tarjeta MicroSD. Esto es útil para cuando se está seguro de que la trayectoria a seguir del brazo va a ser la misma todo el tiempo, entonces como no es necesario cambiarla, la trayectoria se carga una sola vez (o se almacena permanentemente) en la ESP32, independizando la ejecución de la trayectoria del autómatas y de la red Wi-Fi.

9. Funcionamiento

A través del siguiente [link](#) puede observarse el robot serie en funcionamiento, y su interacción coordinada con el robot móvil para la entrega y recepción del vaso de agua.

10. Conclusiones

El desarrollo del manipulador robótico permitió cumplir satisfactoriamente los objetivos planteados por la cátedra, integrando de manera coherente los conocimientos adquiridos en cinemática, dinámica, diseño mecánico y control. El robot logró ejecutar de forma precisa y estable las tareas de manipulación del vaso de agua, validando tanto el modelo matemático como las decisiones constructivas y de selección de actuadores. Del mismo modo, la implementación del controlador local en la ESP32 C3 y del autómatas central para la coordinación con el robot móvil demostró la correcta aplicación de los conceptos de programación orientada a objetos, control distribuido y comunicación mediante MQTT.

La experiencia adquirida a lo largo del proyecto permitió consolidar las herramientas teóricas de la asignatura y llevarlas a una solución funcional, capaz de operar de manera repetible y segura. A futuro, se identifican varias líneas de mejora: incorporar control cerrado con sensado articular para aumentar precisión, añadir modelos internos de trayectoria en la ESP32 para reducir dependencia de la red Wi-Fi, optimizar la estructura para disminuir peso y mejorar rigidez, y explorar métodos más avanzados de planificación y control. Estas extensiones permitirían aumentar la autonomía, robustez y versatilidad del sistema, acercándolo a aplicaciones de mayor complejidad.